SRE Academy: CI/CD with GitHub Actions

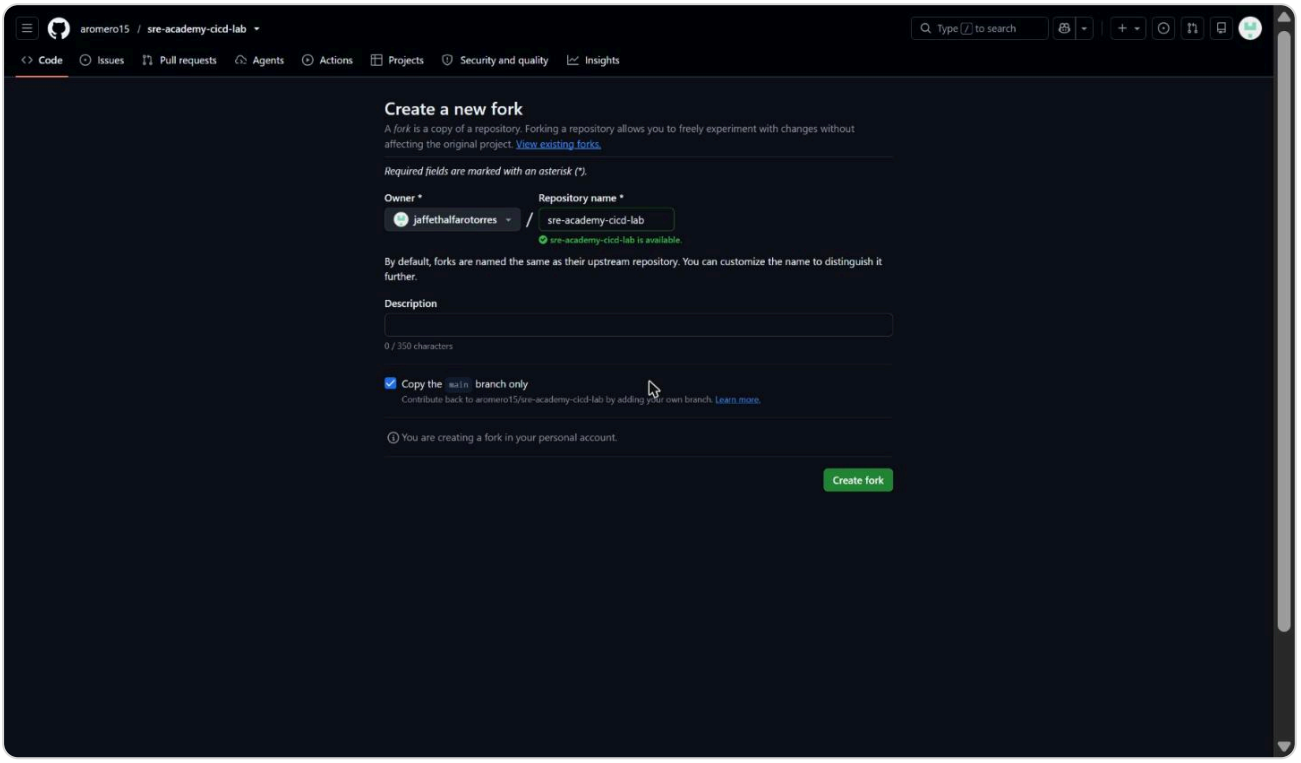
Step-by-step visual guide of the lab, done on the fork `jaffethalfarotorres/sre-academy-cicd-lab` (Oct 21, 2026).

> **Big picture.** `pr.yml` runs the tests on every Pull Request to `main` (a safety net before merging). `ci.yml` runs after merge: test → build (Docker) → staging → production, chained with `needs`. Staging and production are simulated with `echo`.

1 Fork the lab repo

On `aromero15/sre-academy-cicd-lab` click **Fork**. Owner = your personal account, keep the name, leave "Copy the main branch only" ticked, then **Create fork**.

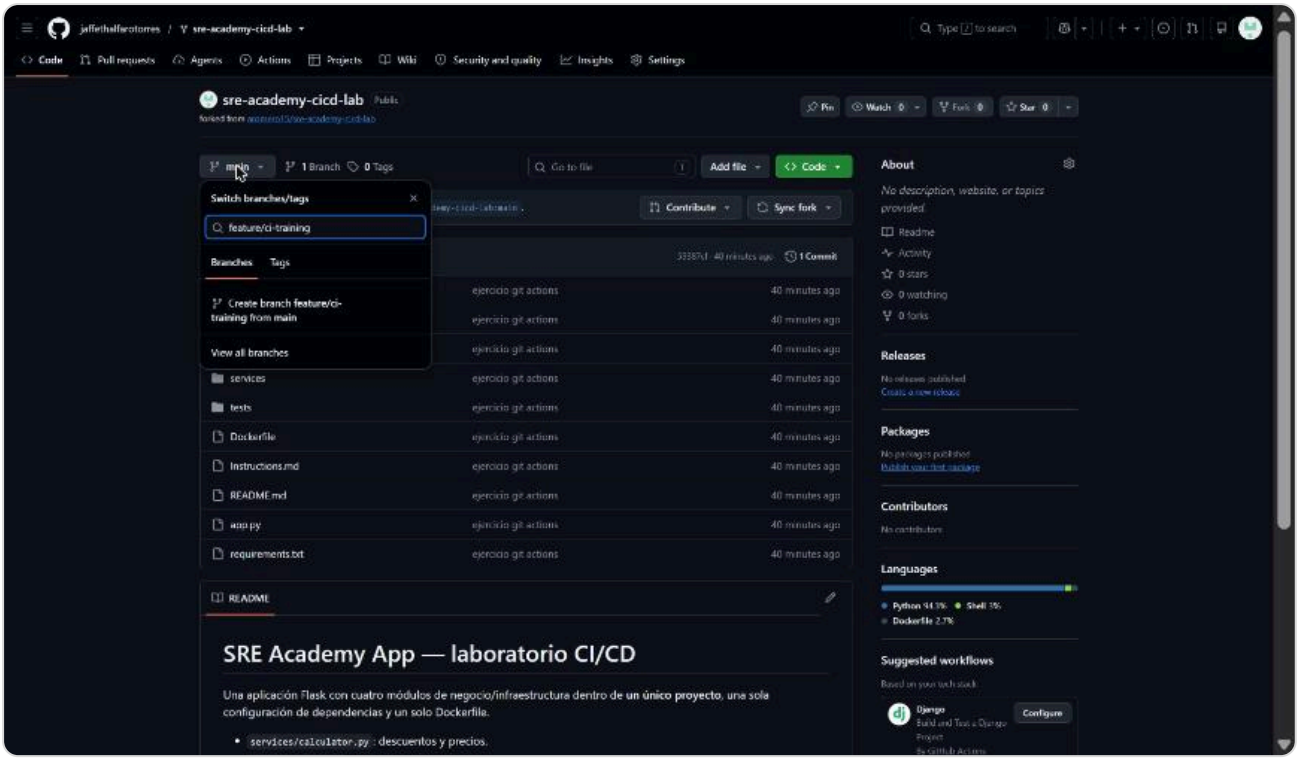
Why: you can't push to the instructor's repo, so you work on your own copy. The note says to fork `sre-academy`, but that repo returned 404, so the lab repo (which already contains the app) was used, as the instructions allow.



2 Create the branch feature/ci-training

In your fork open the branch picker, type `feature/ci-training` and choose “Create branch from main”.

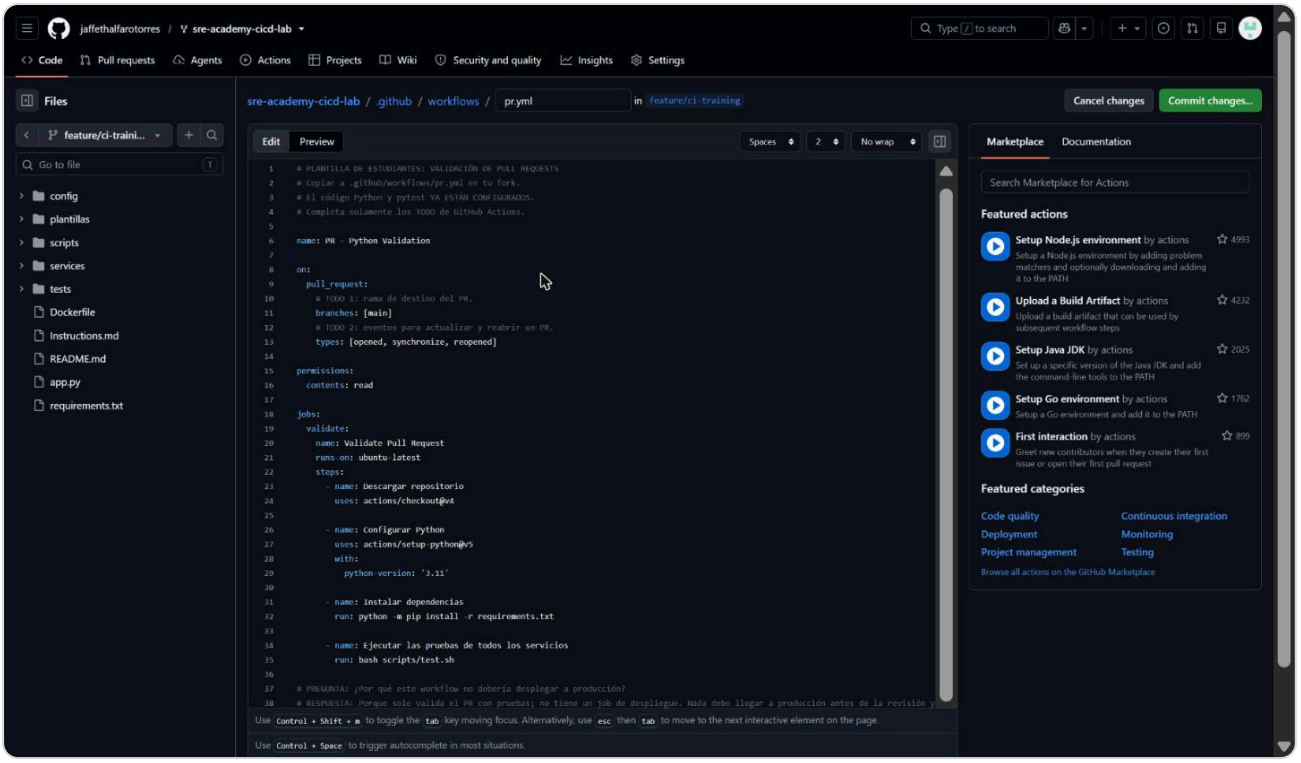
Why: changes go on a feature branch, never straight to main, so they can be reviewed in a PR.



3 Create pr.yml (Activity A)

Add file → Create new file → `.github/workflows/pr.yml`. Fill the TODOs: `branches: [main]` and `types: [opened, synchronize, reopened]`. Install and test steps were already in the template.

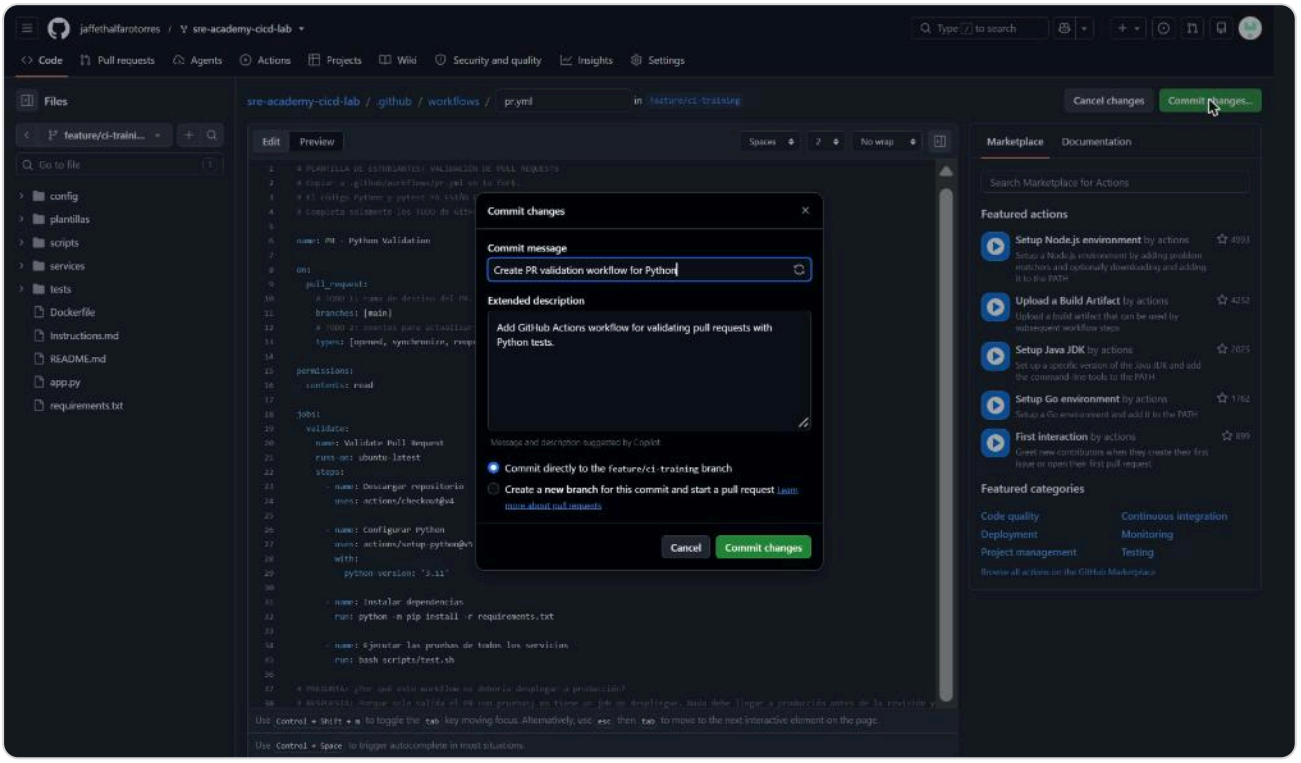
Why: `synchronize` re-runs the check on every new push to the PR; `reopened` covers closed→open.



4 Commit to the feature branch

Commit changes… → keep “Commit directly to the feature/ci-training branch” → Commit.

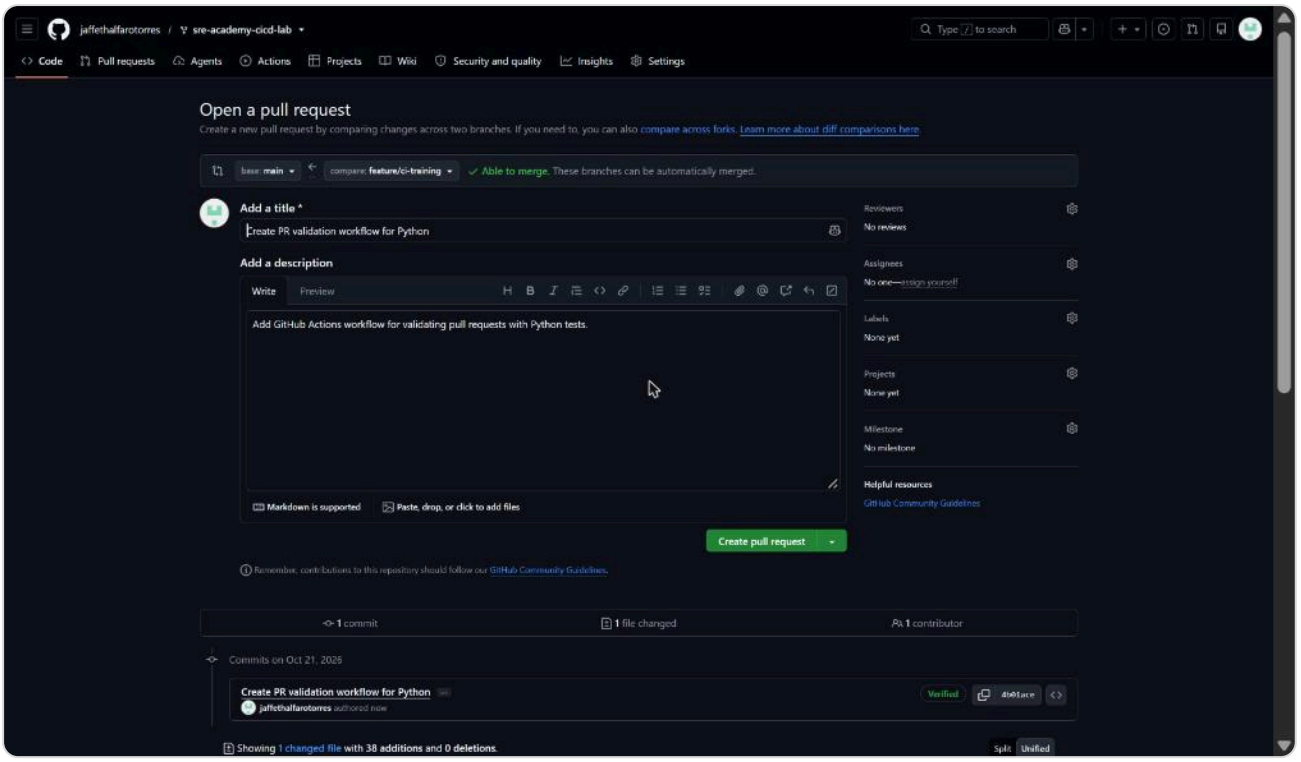
Tip: the dialog pre-fills a Copilot message; overwrite it with your own.



5 Open the Pull Request

Compare page: base `main` ← compare `feature/ci-training`, both in your own fork. Click **Create pull request**.

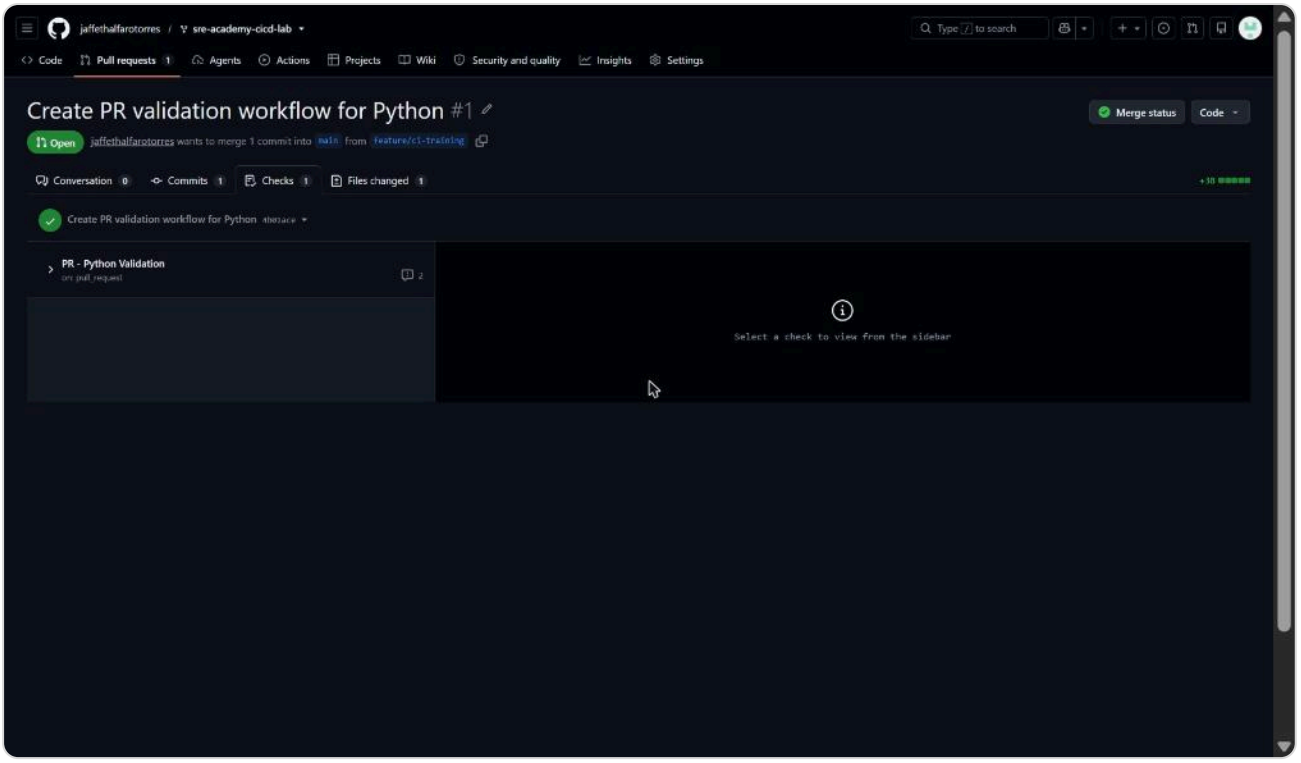
Check: the base must be YOUR fork's main, not the instructor's repo.



6 First check turns green

Open the **Checks** tab: “PR - Python Validation” passes (all 20 tests).

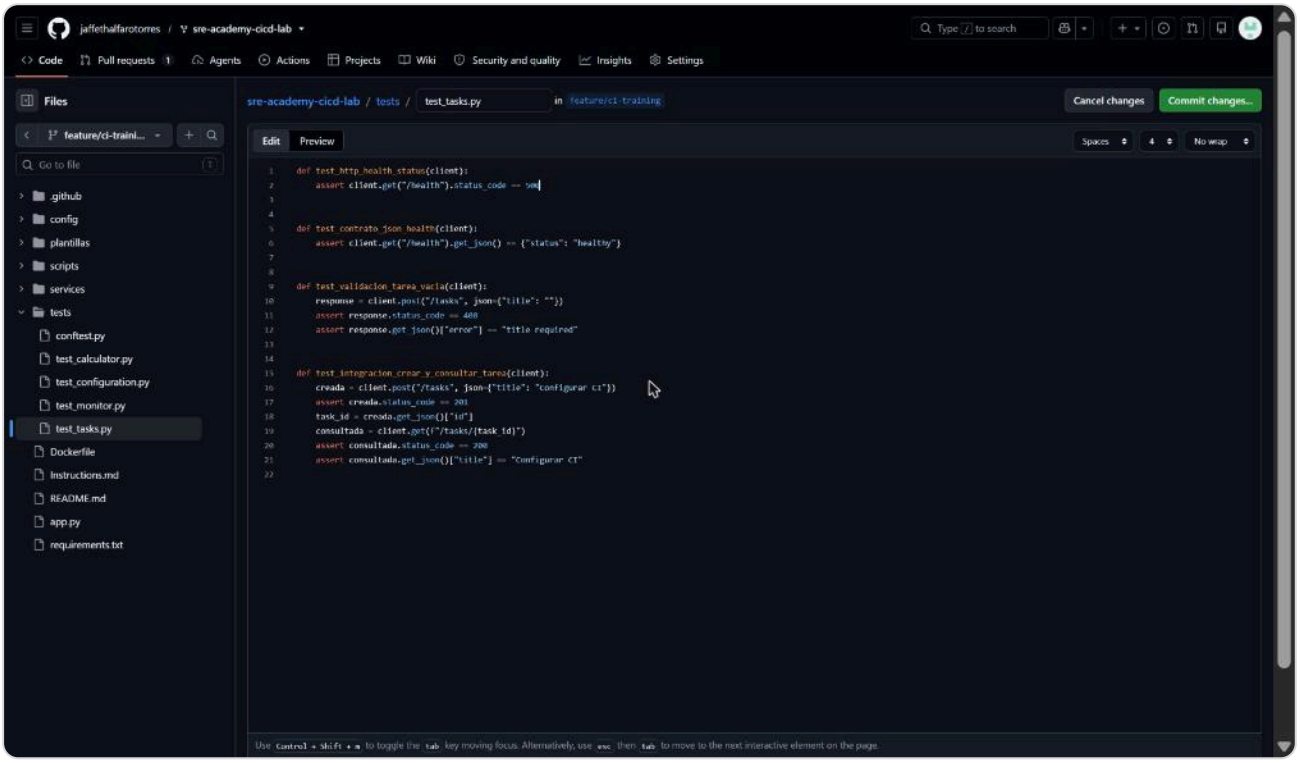
This is the happy path: PR opened → workflow ran → tests green.



7 Failure test: break a test on purpose

Edit `tests/test_tasks.py`, line 2: change `status_code == 200` to `== 500` and commit to the same branch.

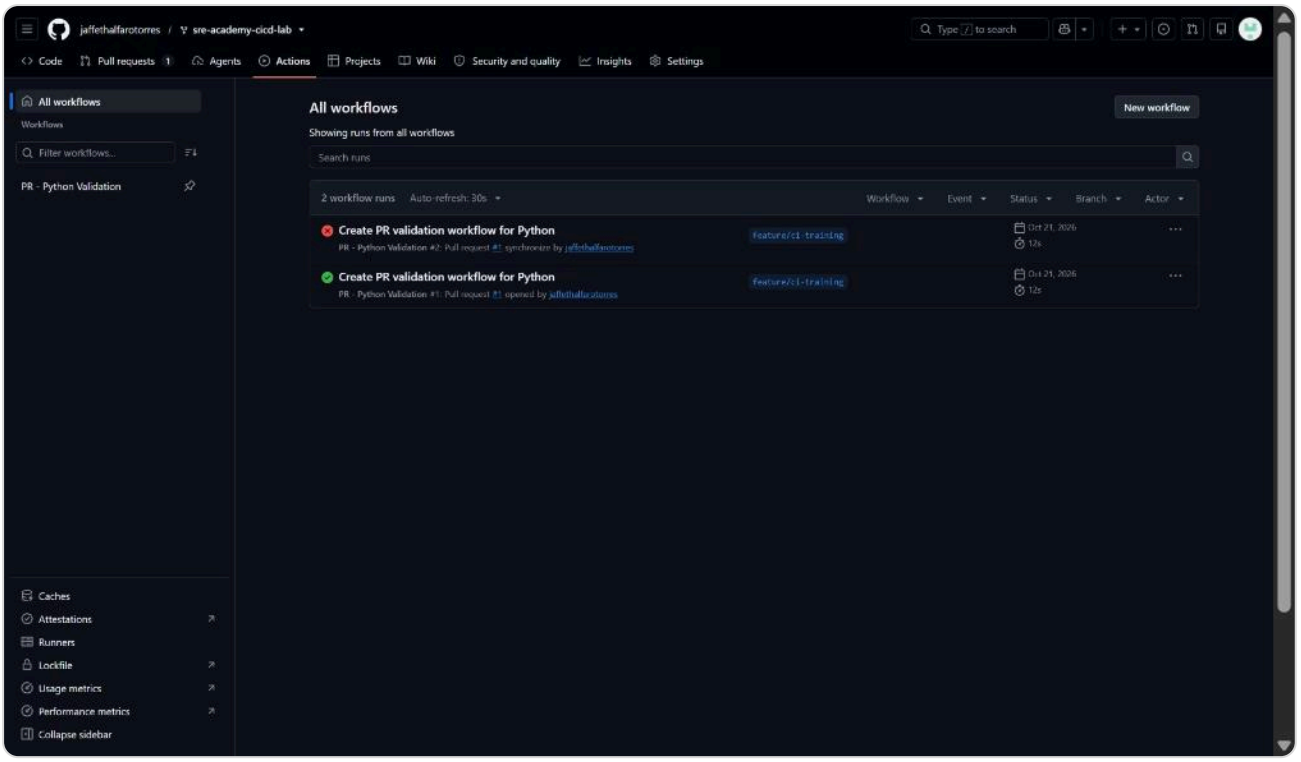
Why: you learn what a failing PR looks like before it happens by accident.



8 See the red run

Actions tab: the new run (#2, event *synchronize*) shows a red × while #1 stays green.

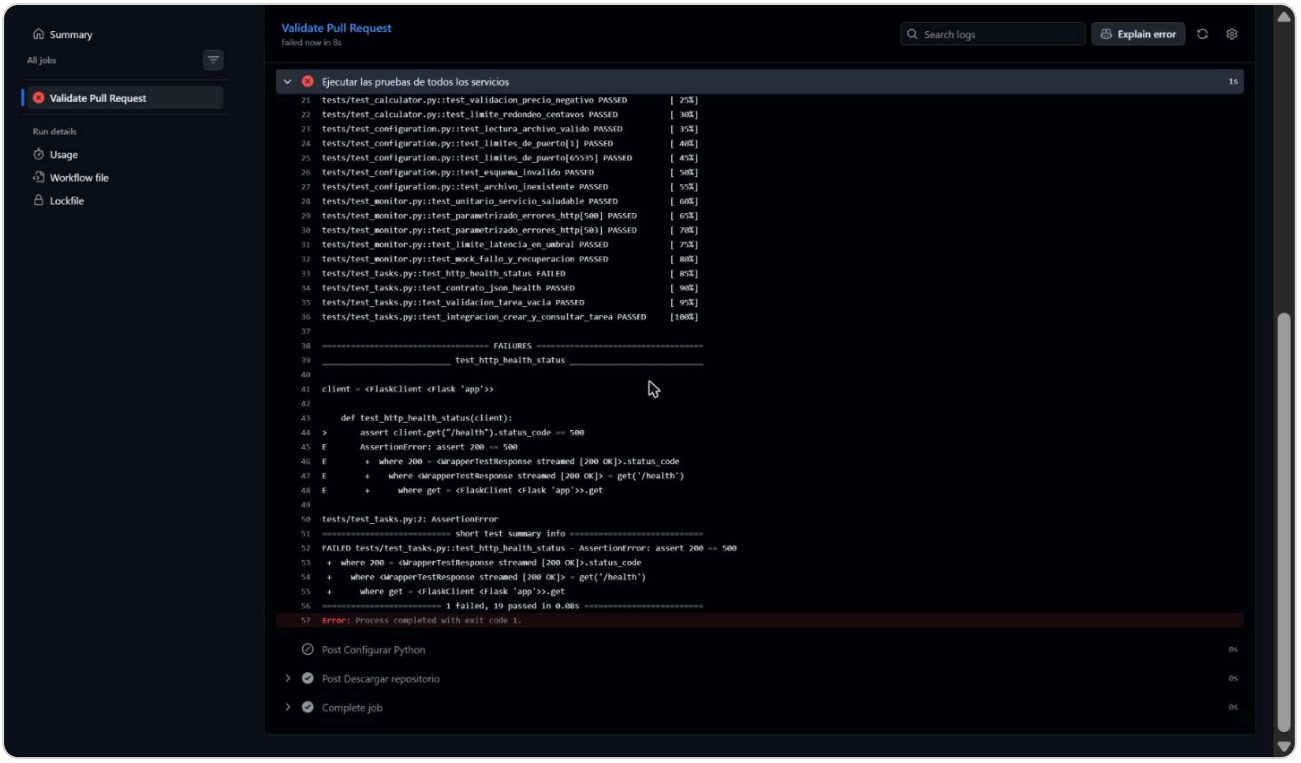
The PR gets blocked/flagged automatically. No human had to notice.



9 Read the logs

Open the run → job → step “Ejecutar las pruebas”. Look for `AssertionError: assert 200 == 500` and “1 failed, 19 passed”.

Reading: the endpoint returned 200 (real), the test expected 500 (wrong). The log names the exact test and line.



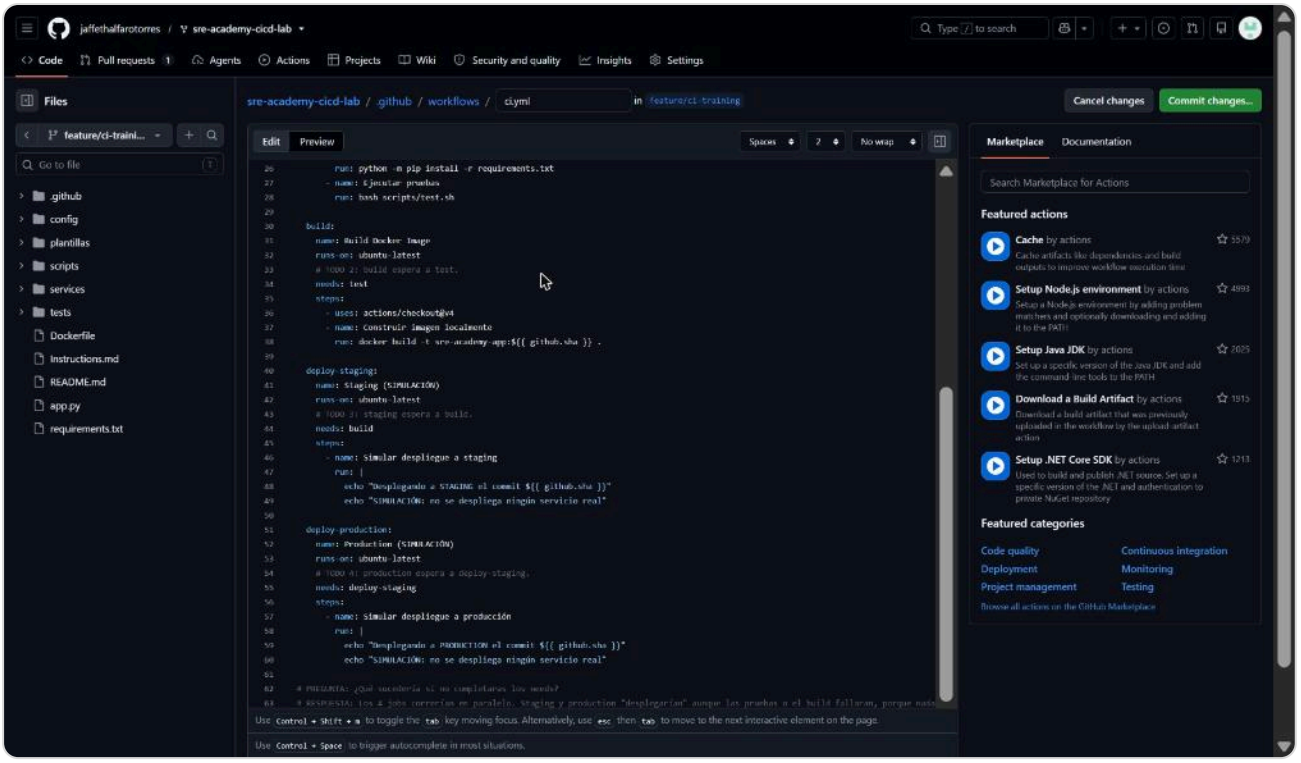
10 Restore 200 and push again

Edit the same line back to `== 200` and commit (“restore health check expectation to 200”). A new run starts and passes.

11 Create ci.yml (Activity B)

Create `.github/workflows/ci.yml` on the same branch: `branches: [main]` and the chain `build: needs: test`, `deploy-staging: needs: build`, `deploy-production: needs: deploy-staging`.

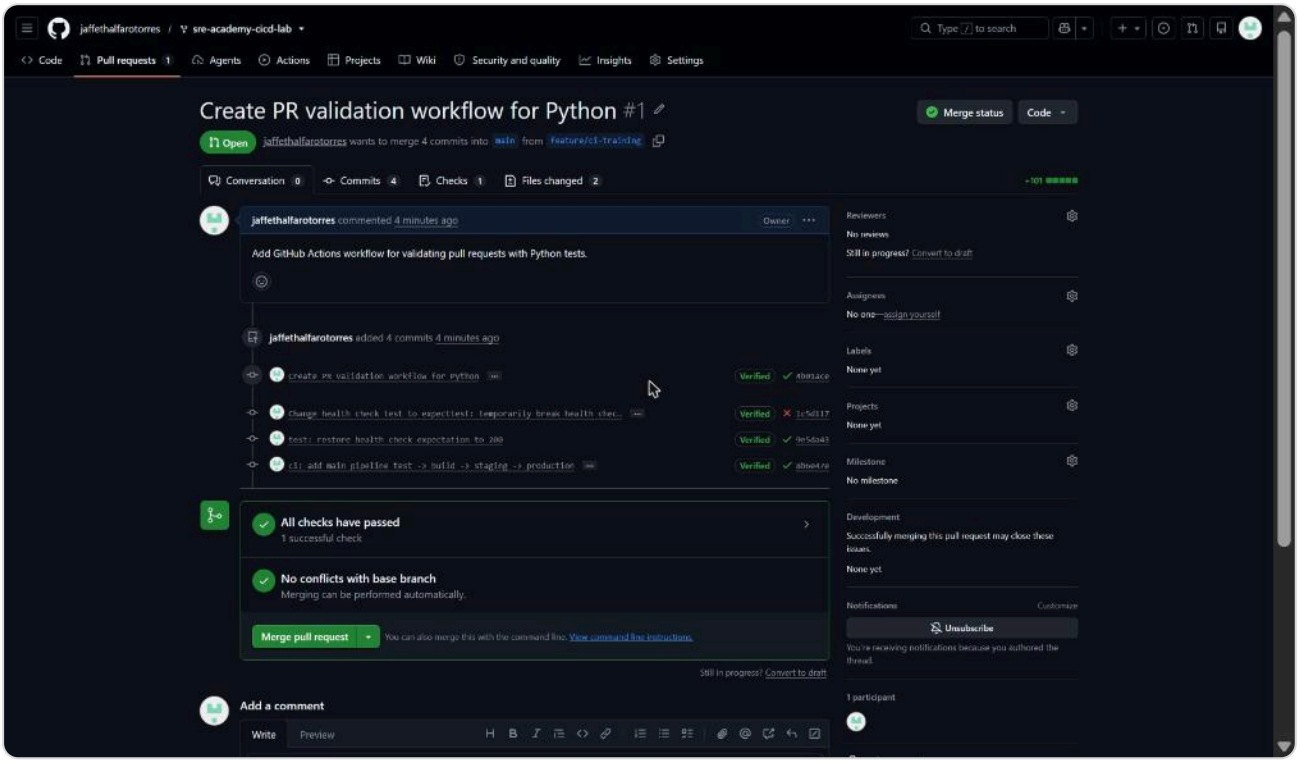
Answer to the lab question: without `needs` all 4 jobs run in parallel, so “production” would deploy even if tests failed.



12 All checks passed → merge

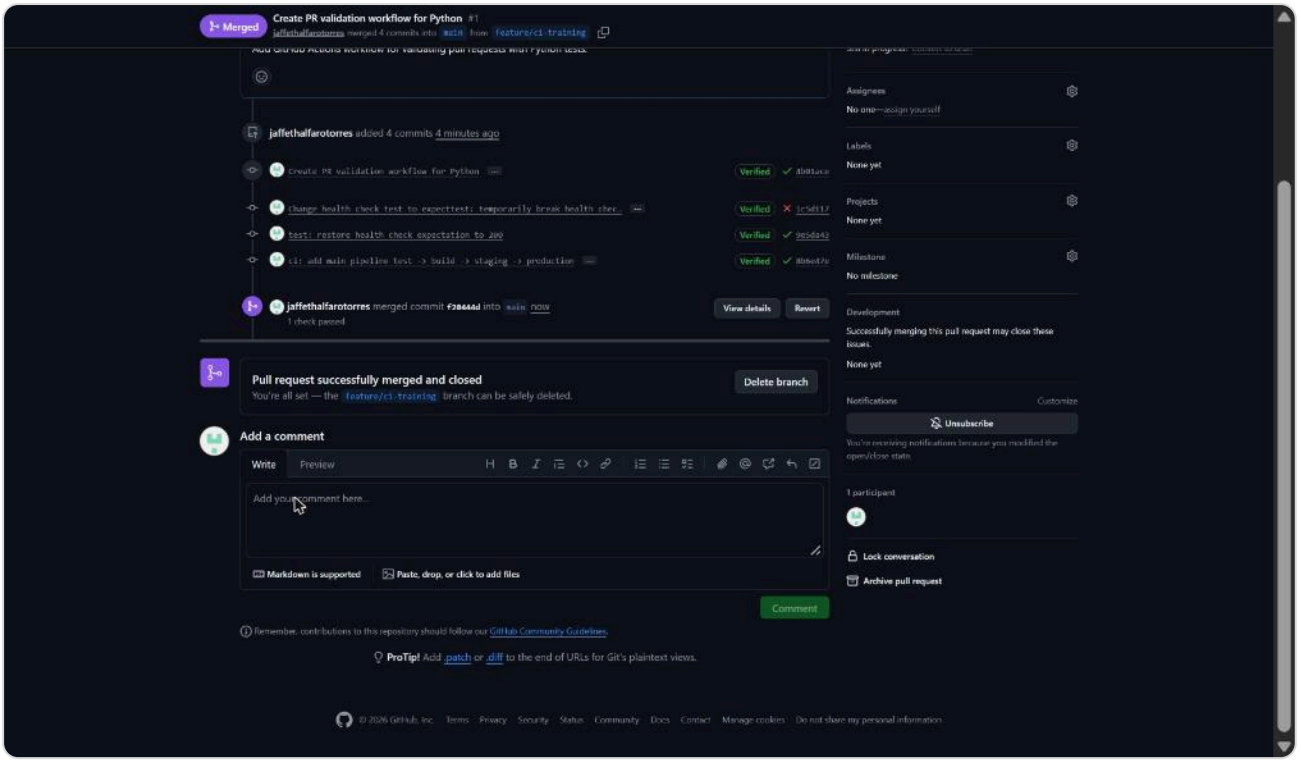
PR shows “All checks have passed” and “No conflicts”. Click **Merge pull request**, then **Confirm merge**.

Only merge when green. That's the whole point of the PR check.



13 PR merged

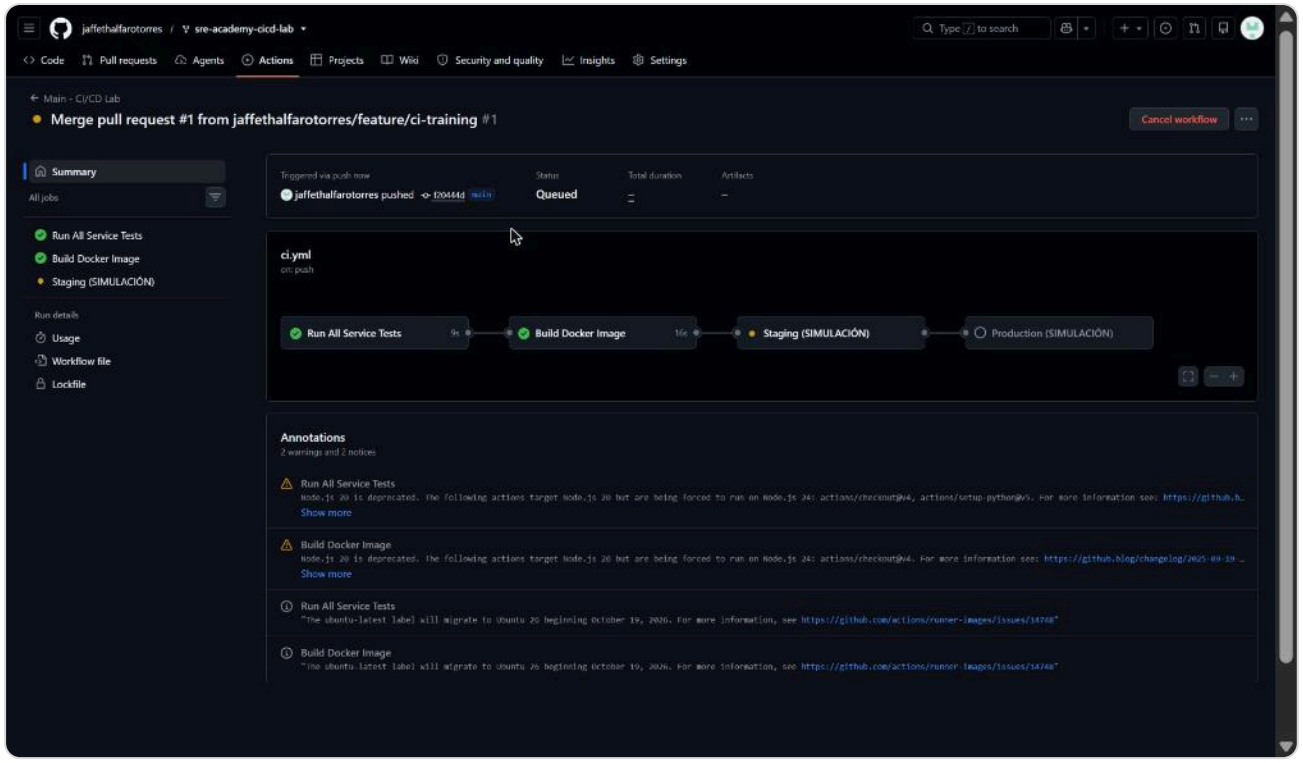
State changes to Merged. The merge is a push to `main`, which triggers `ci.yml`.



14 Pipeline runs in order

Actions → “Main - CI/CD Lab”. The graph shows Tests ✔ → Build ✔ → Staging (running) → Production (waiting).

`needs` in action: each job waits for the previous one and stops if it fails.


jaffethalfarotorres / sre-academy-cicd-lab
Type / to search
Code
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Main - CI/CD Lab
Merge pull request #1 from jaffethalfarotorres/feature/ci-training #1
Cancel workflow
Summary
All jobs
Run All Service Tests
Build Docker Image
Staging (SIMULACIÓN)
Run details
Usage
Workflow file
Lockfile
Triggered via push now
jaffethalfarotorres pushed
Status
Queued
Total duration
Artifacts
ci.yml
on: push
Run All Service Tests
9s
Build Docker Image
16s
Staging (SIMULACIÓN)
Production (SIMULACIÓN)
Annotations
2 warnings and 2 notices
Run All Service Tests
Show more
Build Docker Image
Show more
Run All Service Tests
Build Docker Image

15 Final result

All four jobs green in 42 s: test → build → deploy-staging → deploy-production. Staging and production only run `echo`; nothing real is deployed.

